

Day 2 – JOIN + Aggregation + Analysis

Part 4: JOIN + Aggregation

1. Course title + instructor name + enrollments.

SQLQuery1.sql - EB...BTESAM\ebtes (55))*

```
---part4:--  
--1--  
SELECT  
    c.Title AS CourseTitle,  
    i.FullName AS InstructorName,  
    COUNT(e.EnrollmentID) AS TotalEnrollments  
FROM Courses c  
JOIN Instructors i  
    ON c.InstructorID = i.InstructorID  
LEFT JOIN Enrollments e  
    ON c.CourseID = e.CourseID  
GROUP BY  
    c.CourseID, c.Title, i.FullName  
ORDER BY TotalEnrollments DESC;
```

100 %

Results Messages

	CourseTitle	InstructorName	TotalEnrollments
1	HTML & CSS Basics	Sarah Ahmed	2
2	Python for Data Analysis	Mohammed Al-Busaidi	2
3	JavaScript Advanced	Sarah Ahmed	2
4	Excel for Business	Mohammed Al-Busaidi	1

2. Category name + total courses + average price.

SQLQuery1.sql - EB...BTESAM\ebtes (55))*

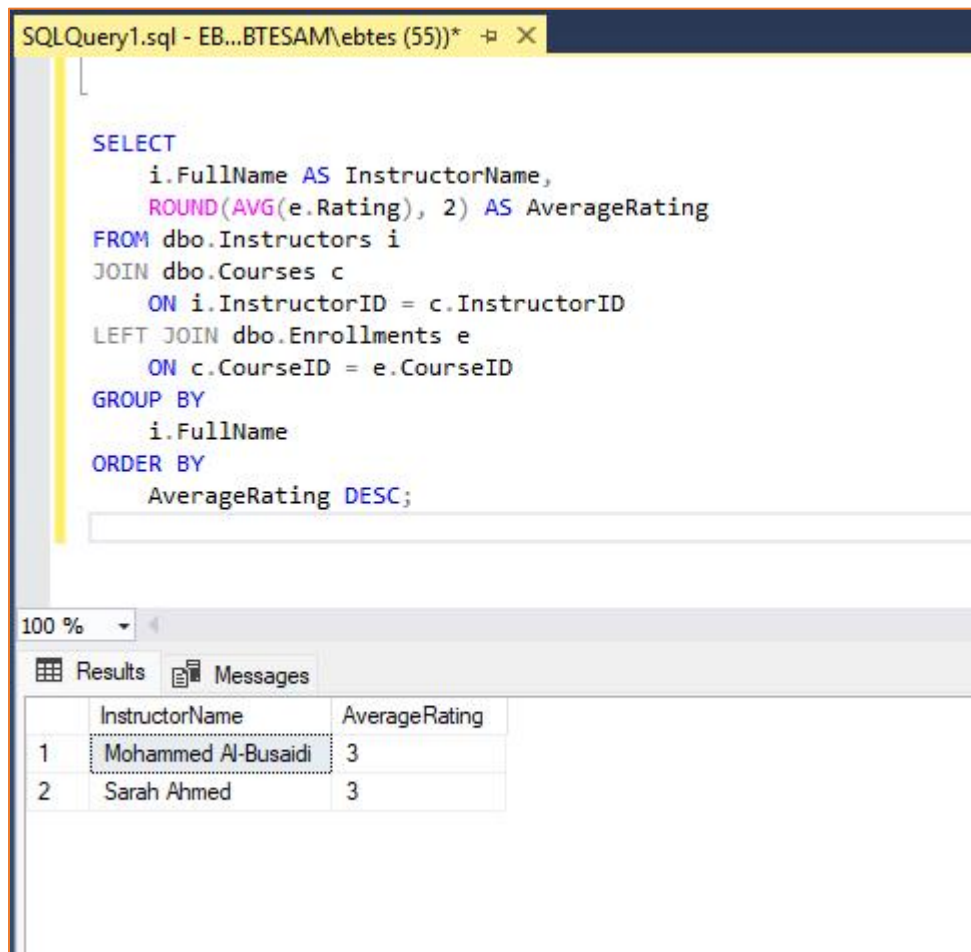
```
--2--  
SELECT  
    cat.CategoryName,  
    COUNT(c.CourseID) AS TotalCourses,  
    AVG(c.Price) AS AveragePrice  
FROM dbo.Categories cat  
LEFT JOIN dbo.Courses c  
    ON cat.CategoryID = c.CategoryID  
GROUP BY  
    cat.CategoryName  
ORDER BY  
    cat.CategoryName;
```

100 %

Results Messages

	CategoryName	TotalCourses	AveragePrice
1	Business	1	19.990000
2	Data Science	1	49.990000
3	Web Development	2	34.990000

3. Instructor name + average course rating.



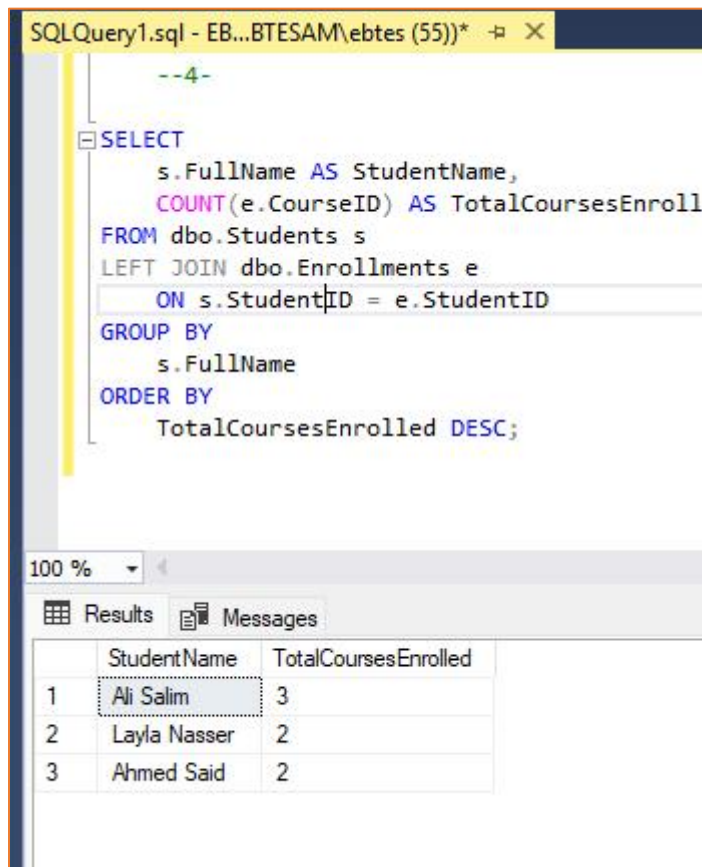
The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query in the 'SQLQuery1.sql' file. The query is designed to retrieve instructor names and their average course ratings, rounded to two decimal places. It uses a series of joins to connect the 'Instructors' table with 'Courses' and 'Enrollments' tables. The results are grouped by instructor name and ordered by their average rating in descending order.

```
SELECT
    i.FullName AS InstructorName,
    ROUND(AVG(e.Rating), 2) AS AverageRating
FROM dbo.Instructors i
JOIN dbo.Courses c
    ON i.InstructorID = c.InstructorID
LEFT JOIN dbo.Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY
    i.FullName
ORDER BY
    AverageRating DESC;
```

The bottom pane shows the 'Results' tab with a table containing two rows of data. The first row shows 'Mohammed Al-Busaidi' with an average rating of 3. The second row shows 'Sarah Ahmed' with an average rating of 3. The table has two columns: 'InstructorName' and 'AverageRating'.

	InstructorName	AverageRating
1	Mohammed Al-Busaidi	3
2	Sarah Ahmed	3

4. Student name + total courses enrolled.



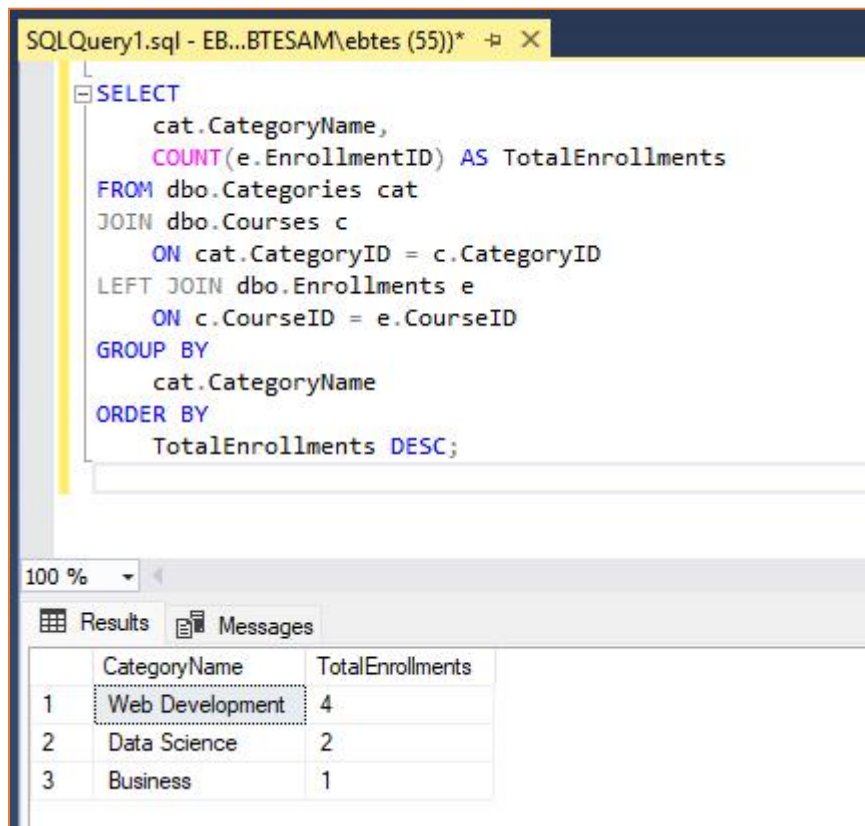
The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

```
--4-  
  
SELECT  
    s.FullName AS StudentName,  
    COUNT(e.CourseID) AS TotalCoursesEnroll  
FROM    dbo.Students s  
LEFT JOIN    dbo.Enrollments e  
    ON s.StudentID = e.StudentID  
GROUP BY  
    s.FullName  
ORDER BY  
    TotalCoursesEnrolled DESC;
```

Below the query editor, the "Results" tab is active, displaying the following data:

	StudentName	TotalCoursesEnrolled
1	Ali Salim	3
2	Layla Nasser	2
3	Ahmed Said	2

5. Category name + total enrollments.



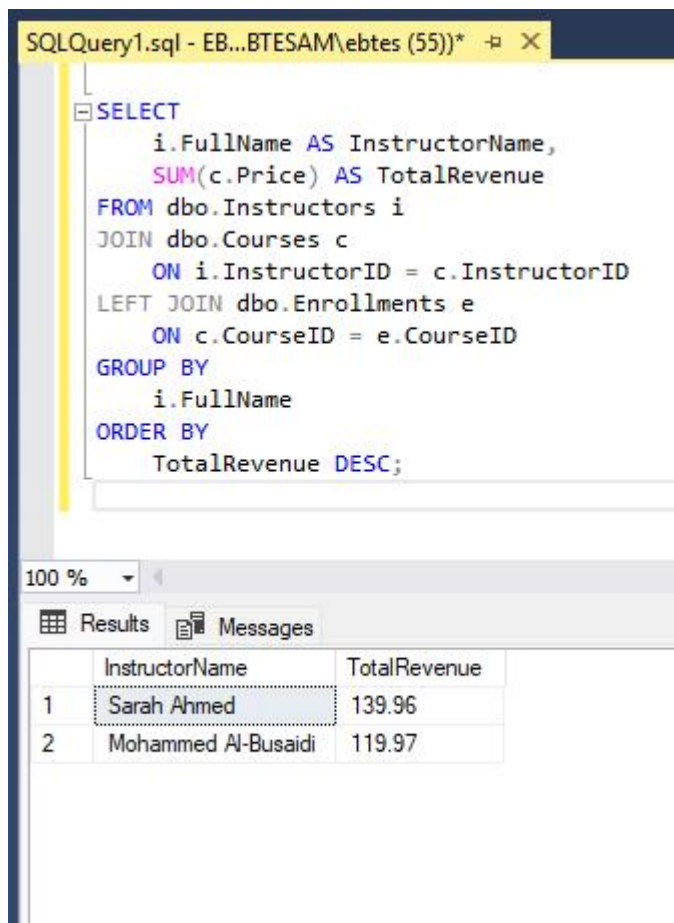
The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

```
SELECT
    cat.CategoryName,
    COUNT(e.EnrollmentID) AS TotalEnrollments
FROM dbo.Categories cat
JOIN dbo.Courses c
    ON cat.CategoryID = c.CategoryID
LEFT JOIN dbo.Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY
    cat.CategoryName
ORDER BY
    TotalEnrollments DESC;
```

Below the query editor, the "Results" tab is active, displaying a table with 3 rows and 2 columns: "CategoryName" and "TotalEnrollments".

	CategoryName	TotalEnrollments
1	Web Development	4
2	Data Science	2
3	Business	1

6. Instructor name + total revenue.



The screenshot displays the SQL Server Enterprise Manager interface. At the top, a query window titled 'SQLQuery1.sql - EB...BTESAM\ebtes (55))*' contains the following SQL code:

```
SELECT
    i.FullName AS InstructorName,
    SUM(c.Price) AS TotalRevenue
FROM dbo.Instructors i
JOIN dbo.Courses c
    ON i.InstructorID = c.InstructorID
LEFT JOIN dbo.Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY
    i.FullName
ORDER BY
    TotalRevenue DESC;
```

Below the query editor, the 'Results' tab is active, showing a table with two columns: 'InstructorName' and 'TotalRevenue'. The table contains two rows of data, ordered by total revenue in descending order.

	InstructorName	TotalRevenue
1	Sarah Ahmed	139.96
2	Mohammed Al-Busaidi	119.97

7. Course title + % of students completed 100%.

SQLQuery1.sql - EB...BTESAM\ebtes (55))

```
SELECT
    c.Title AS CourseTitle,
    CAST(
        100.0 * SUM(CASE WHEN e.CompletionPercent = 100 THEN 1 ELSE 0 END)
        / NULLIF(COUNT(e.EnrollmentID), 0)
        AS DECIMAL(5,2)
    ) AS CompletionPercentage
FROM dbo.Courses c
LEFT JOIN dbo.Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY
    c.Title
ORDER BY
    CompletionPercentage DESC;
```

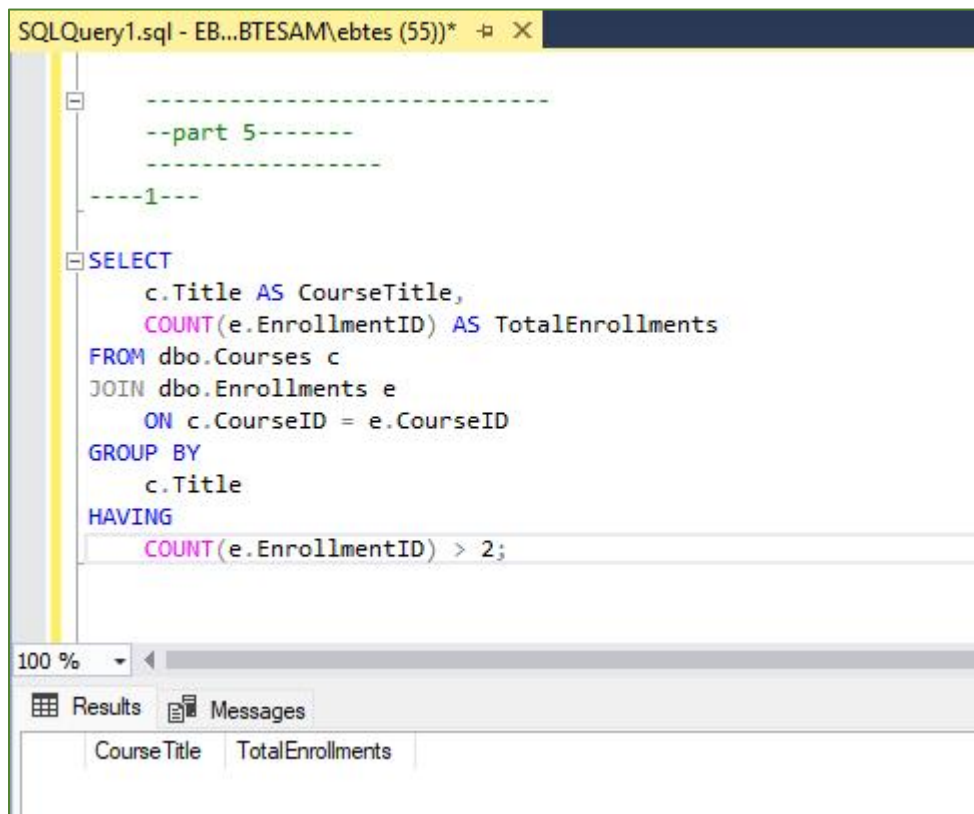
100 %

Results Messages

	CourseTitle	CompletionPercentage
1	HTML & CSS Basics	50.00
2	JavaScript Advanced	0.00
3	Python for Data Analysis	0.00
4	Excel for Business	0.00

Part 5: HAVING Practice

1. Courses with more than 1 enrollments.



The screenshot shows a SQL Server Enterprise Manager window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query window contains the following SQL code:

```
-----  
--part 5-----  
-----  
----1---  
  
SELECT  
    c.Title AS CourseTitle,  
    COUNT(e.EnrollmentID) AS TotalEnrollments  
FROM  dbo.Courses c  
JOIN  dbo.Enrollments e  
      ON c.CourseID = e.CourseID  
GROUP BY  
    c.Title  
HAVING  
    COUNT(e.EnrollmentID) > 2;
```

Below the query window, there is a toolbar with "Results" and "Messages" buttons. The "Results" button is active, and a grid is visible with the following columns:

CourseTitle	TotalEnrollments
-------------	------------------

2. Instructors with average rating above 2.

SQLQuery1.sql - EB...BTESAM\ebtes (55))*

```
---2---  
  
SELECT  
    i.FullName AS InstructorName,  
    AVG(e.Rating) AS AverageRating  
FROM    dbo.Instructors i  
JOIN    dbo.Courses c  
        ON i.InstructorID = c.InstructorID  
JOIN    dbo.Enrollments e  
        ON c.CourseID = e.CourseID  
GROUP BY  
    i.FullName  
HAVING  
    AVG(e.Rating) > 4;
```

100 %

Results Messages

InstructorName	AverageRating
----------------	---------------

3. Courses with average completion below 60%.

SQLQuery1.sql - EB...BTESAM\ebtes (55))*

```
--3---  
  
SELECT  
    c.Title AS CourseTitle,  
    AVG(e.CompletionPercent) AS AvgCompletion  
FROM    dbo.Courses c  
JOIN    dbo.Enrollments e  
        ON c.CourseID = e.CourseID  
GROUP BY  
    c.Title  
HAVING  
    AVG(e.CompletionPercent) < 60;
```

100 %

Results Messages

	CourseTitle	AvgCompletion
1	JavaScript Advanced	45

4. Categories with more than 1 course.

The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

```
--4--  
  
SELECT  
    cat.CategoryName,  
    COUNT(c.CourseID) AS TotalCourses  
FROM    dbo.Categories cat  
JOIN    dbo.Courses c  
    ON  cat.CategoryID = c.CategoryID  
GROUP BY  
    cat.CategoryName  
HAVING  
    COUNT(c.CourseID) > 1;
```

Below the query, the "Results" tab is active, displaying a table with two columns: "CategoryName" and "TotalCourses".

	CategoryName	TotalCourses
1	Web Development	2

5. Students enrolled in at least 2 courses.

The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

```
--5--  
  
SELECT  
    s.FullName AS StudentName,  
    COUNT(e.CourseID) AS TotalCourses  
FROM    dbo.Students s  
JOIN    dbo.Enrollments e  
    ON  s.StudentID = e.StudentID  
GROUP BY  
    s.FullName  
HAVING  
    COUNT(e.CourseID) >= 2;
```

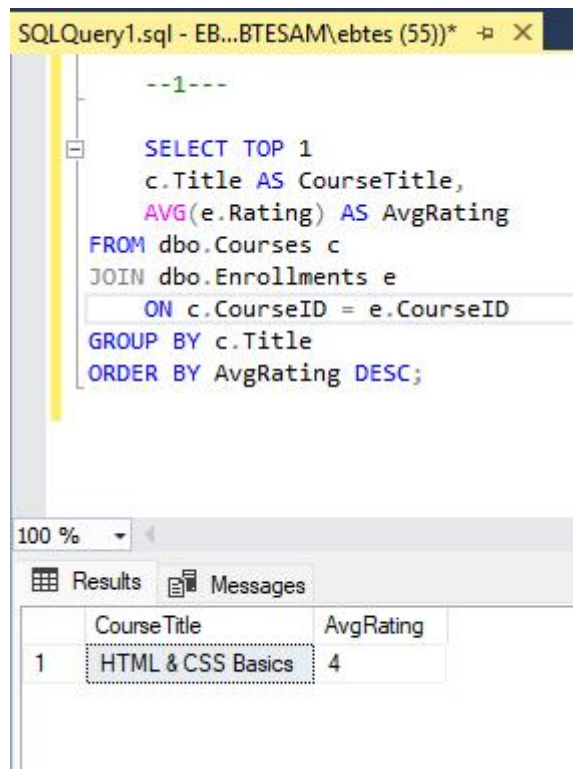
Below the query, the "Results" tab is active, displaying a table with two columns: "StudentName" and "TotalCourses".

	StudentName	TotalCourses
1	Ahmed Said	2
2	Ali Salim	3
3	Layla Nasser	2

Part 6: Analytical Thinking

Answer using SQL + short explanation:

1. Best performing course.



The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))*". The query is as follows:

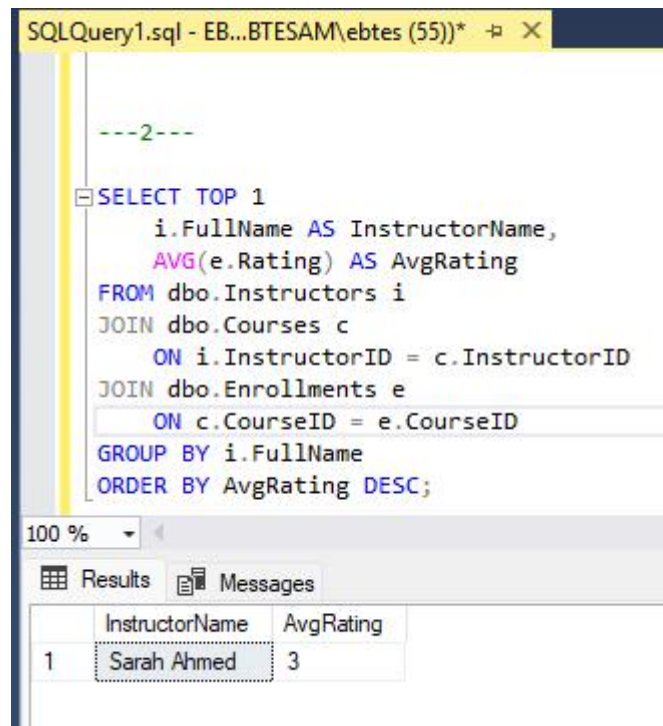
```
--1---  
  
SELECT TOP 1  
  c.Title AS CourseTitle,  
  AVG(e.Rating) AS AvgRating  
FROM  dbo.Courses c  
JOIN  dbo.Enrollments e  
      ON c.CourseID = e.CourseID  
GROUP BY c.Title  
ORDER BY AvgRating DESC;
```

Below the query window, the "Results" tab is active, displaying a table with two columns: "CourseTitle" and "AvgRating". The table contains one row:

	CourseTitle	AvgRating
1	HTML & CSS Basics	4

Finds the course with the highest average rating across all enrollments.

2. Instructor to promote.



The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

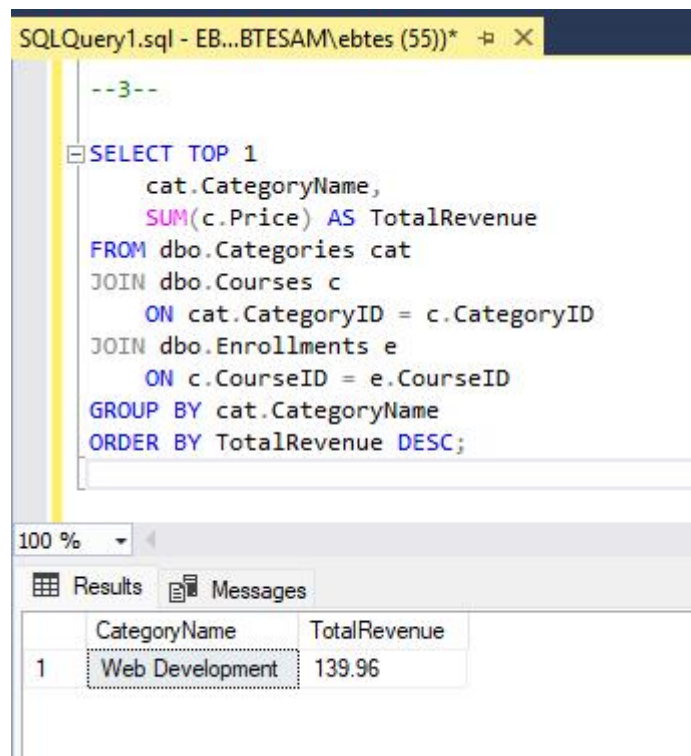
```
---2---  
  
SELECT TOP 1  
    i.FullName AS InstructorName,  
    AVG(e.Rating) AS AvgRating  
FROM    dbo.Instructors i  
JOIN    dbo.Courses c  
    ON  i.InstructorID = c.InstructorID  
JOIN    dbo.Enrollments e  
    ON  c.CourseID = e.CourseID  
GROUP BY i.FullName  
ORDER BY AvgRating DESC;
```

Below the query, the "Results" tab is active, displaying a single row of data:

	InstructorName	AvgRating
1	Sarah Ahmed	3

The instructor whose courses have the highest average student rating.

3. Highest revenue category.



The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is as follows:

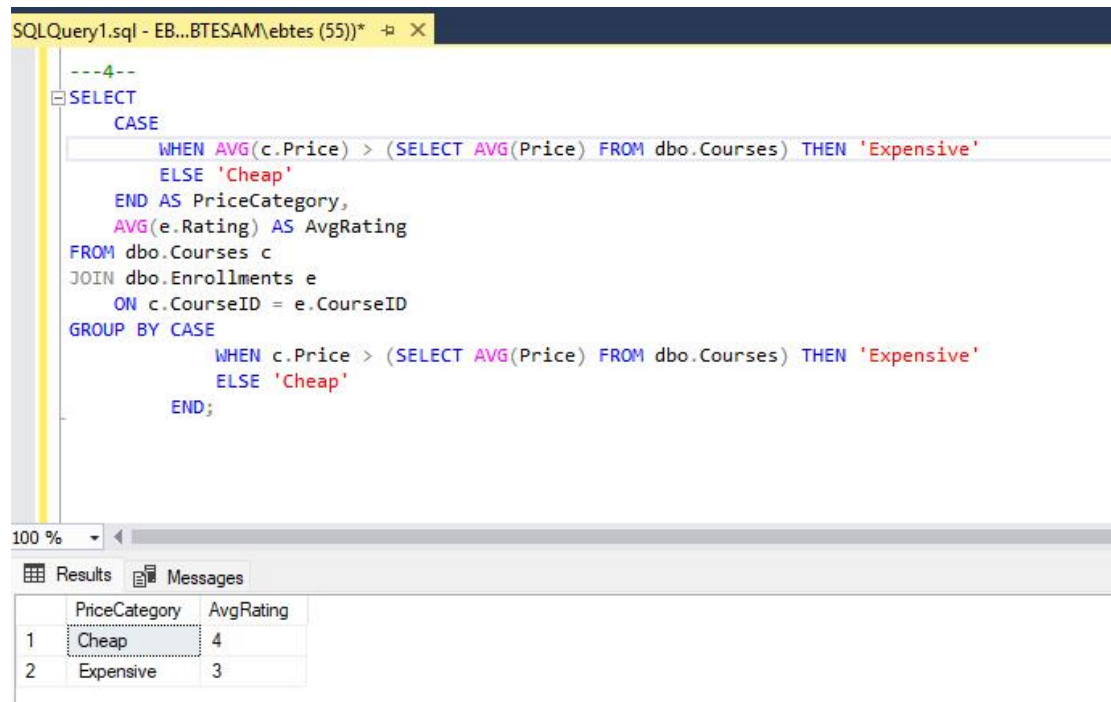
```
--3--  
  
SELECT TOP 1  
    cat.CategoryName,  
    SUM(c.Price) AS TotalRevenue  
FROM  dbo.Categories cat  
JOIN  dbo.Courses c  
    ON cat.CategoryID = c.CategoryID  
JOIN  dbo.Enrollments e  
    ON c.CourseID = e.CourseID  
GROUP BY cat.CategoryName  
ORDER BY TotalRevenue DESC;
```

Below the query window, the "Results" tab is active, displaying a single row of data:

	CategoryName	TotalRevenue
1	Web Development	139.96

Sums course price × enrollments per category to find the highest revenue.

4. Do expensive courses have better ratings?



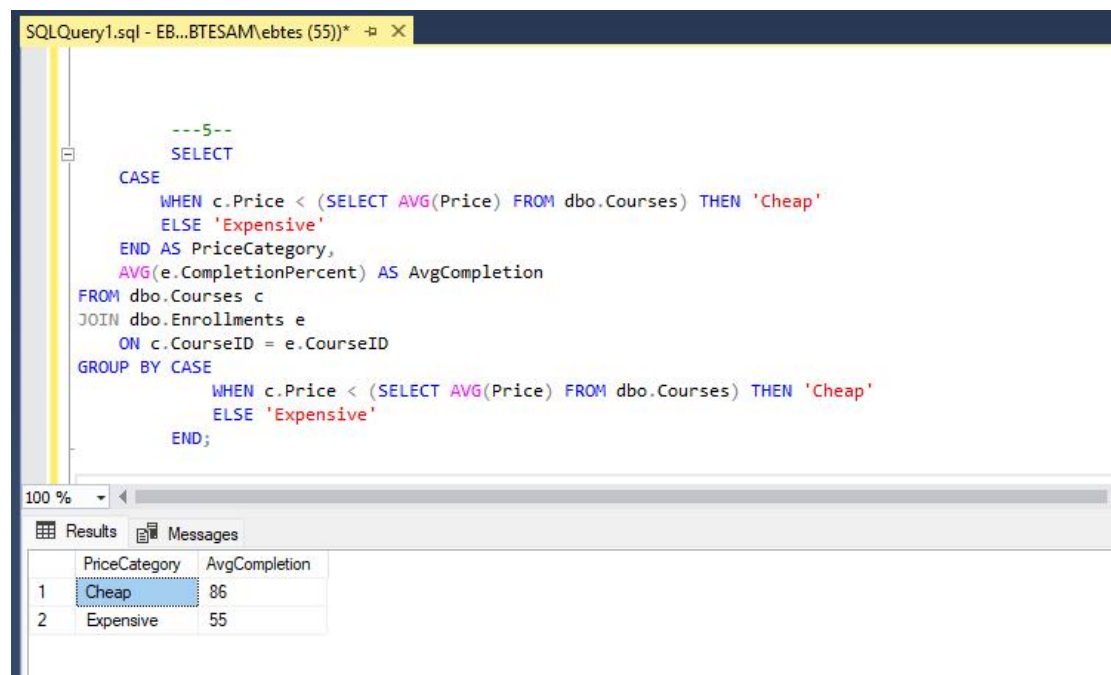
The screenshot shows a SQL query in SQL Server Enterprise Manager. The query is designed to compare the average rating of courses categorized as 'Expensive' versus 'Cheap'. It uses a CASE statement to categorize courses based on their price relative to the overall average price of all courses. The results pane shows two rows: 'Cheap' with an average rating of 4, and 'Expensive' with an average rating of 3.

```
---4---
SELECT
    CASE
        WHEN AVG(c.Price) > (SELECT AVG(Price) FROM dbo.Courses) THEN 'Expensive'
        ELSE 'Cheap'
    END AS PriceCategory,
    AVG(e.Rating) AS AvgRating
FROM dbo.Courses c
JOIN dbo.Enrollments e
    ON c.CourseID = e.CourseID
GROUP BY CASE
    WHEN c.Price > (SELECT AVG(Price) FROM dbo.Courses) THEN 'Expensive'
    ELSE 'Cheap'
END;
```

	PriceCategory	AvgRating
1	Cheap	4
2	Expensive	3

Compares **average rating** of courses above vs below the overall average price. Shows if expensive courses are rated better.

5. Do cheaper courses have higher completion?



The screenshot displays a SQL query in the 'SQLQuery1.sql' window. The query is a SELECT statement with a CASE expression for 'PriceCategory' and an aggregate function 'AVG' for 'AvgCompletion'. It joins 'dbo.Courses' and 'dbo.Enrollments' on 'CourseID'. The results pane shows two rows: 'Cheap' with an average completion of 86, and 'Expensive' with an average completion of 55.

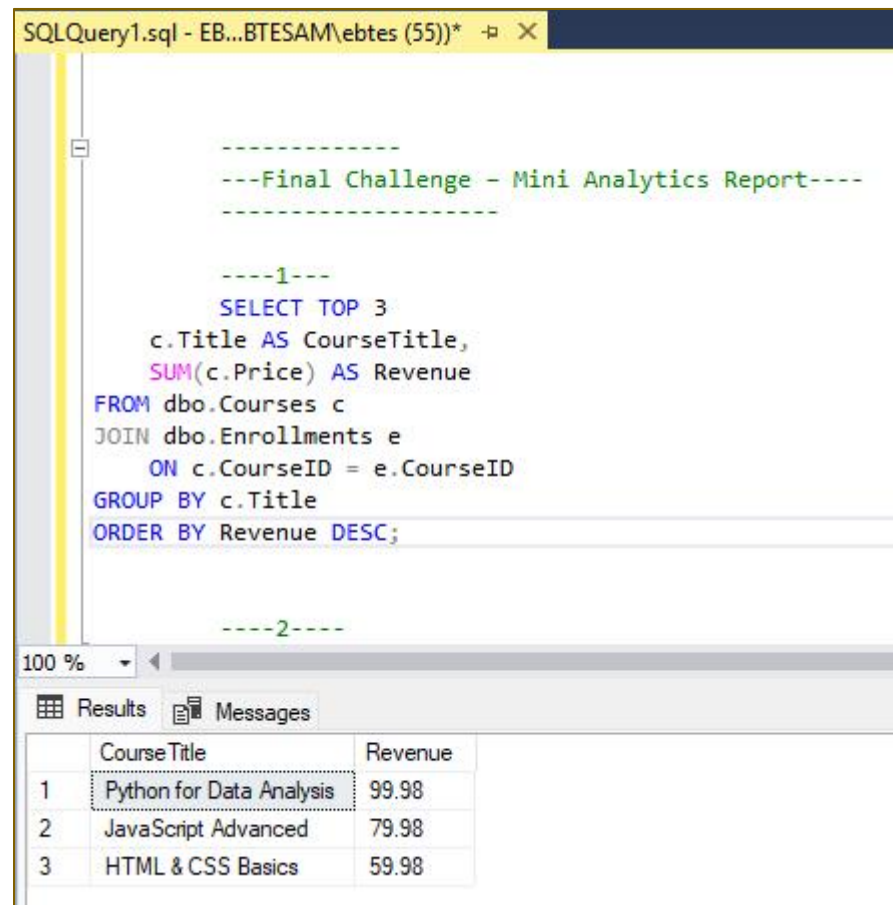
```
SQLQuery1.sql - EB...BTESAM\ebtes (55) *  
  
--5--  
SELECT  
CASE  
    WHEN c.Price < (SELECT AVG(Price) FROM dbo.Courses) THEN 'Cheap'  
    ELSE 'Expensive'  
END AS PriceCategory,  
AVG(e.CompletionPercent) AS AvgCompletion  
FROM dbo.Courses c  
JOIN dbo.Enrollments e  
    ON c.CourseID = e.CourseID  
GROUP BY CASE  
    WHEN c.Price < (SELECT AVG(Price) FROM dbo.Courses) THEN 'Cheap'  
    ELSE 'Expensive'  
END;
```

	PriceCategory	AvgCompletion
1	Cheap	86
2	Expensive	55

Shows if cheaper courses have higher completion percentages compared to expensive ones.

Final Challenge – Mini Analytics Report

1. Top 3 courses by revenue.



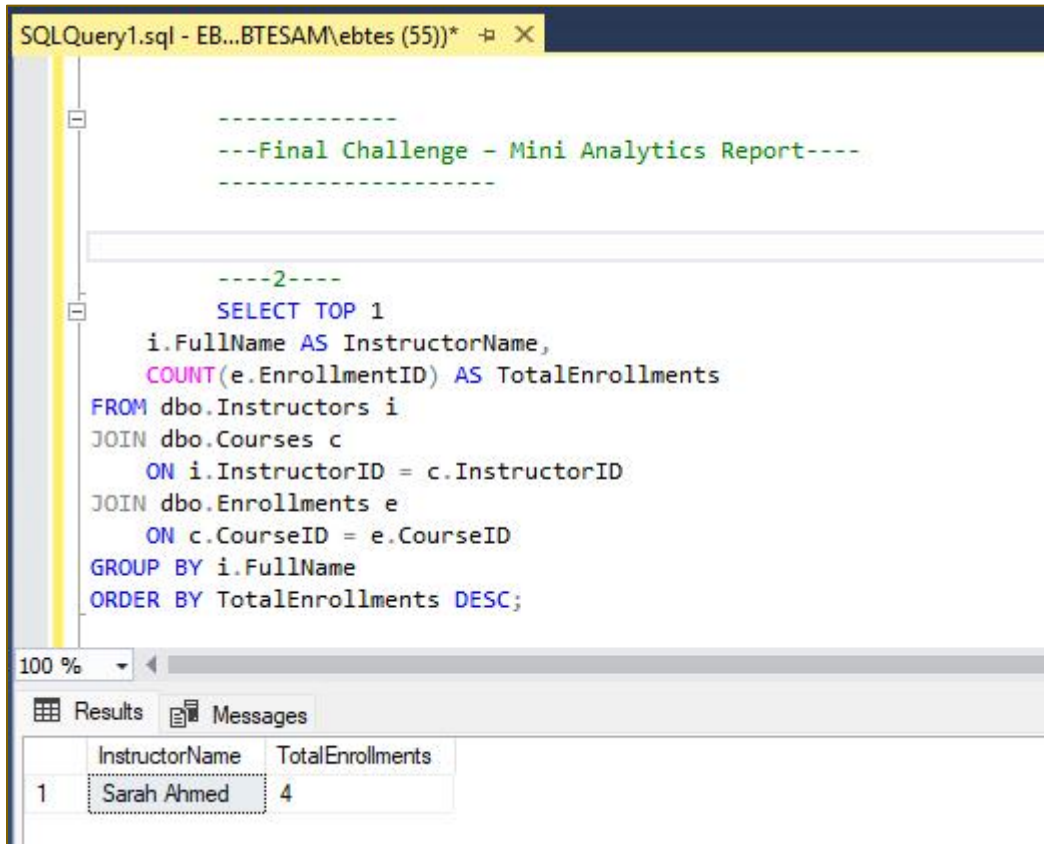
The screenshot shows a SQL Server Enterprise Manager window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))*". The query editor contains the following SQL code:

```
-----  
---Final Challenge - Mini Analytics Report---  
-----  
  
-----1---  
SELECT TOP 3  
    c.Title AS CourseTitle,  
    SUM(c.Price) AS Revenue  
FROM    dbo.Courses c  
JOIN    dbo.Enrollments e  
        ON c.CourseID = e.CourseID  
GROUP BY c.Title  
ORDER BY Revenue DESC;  
  
-----2-----
```

Below the query editor, the "Results" tab is active, displaying the following data:

	CourseTitle	Revenue
1	Python for Data Analysis	99.98
2	JavaScript Advanced	79.98
3	HTML & CSS Basics	59.98

2. Instructor with most enrollments.



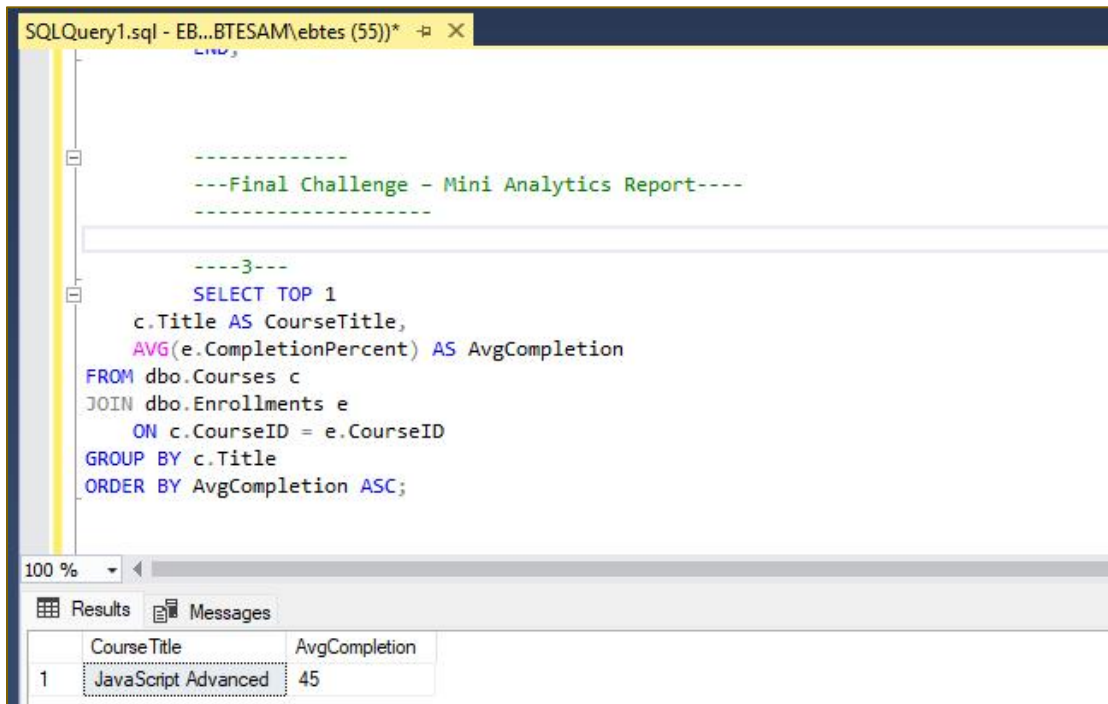
The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is a T-SQL statement that selects the top 1 instructor by the total number of enrollments. The query text is as follows:

```
-----  
---Final Challenge - Mini Analytics Report---  
-----  
  
-----2-----  
SELECT TOP 1  
    i.FullName AS InstructorName,  
    COUNT(e.EnrollmentID) AS TotalEnrollments  
FROM    dbo.Instructors i  
JOIN    dbo.Courses c  
        ON i.InstructorID = c.InstructorID  
JOIN    dbo.Enrollments e  
        ON c.CourseID = e.CourseID  
GROUP BY i.FullName  
ORDER BY TotalEnrollments DESC;
```

Below the query editor, the "Results" tab is active, displaying a single row of data:

	InstructorName	TotalEnrollments
1	Sarah Ahmed	4

3. Course with lowest completion rate.



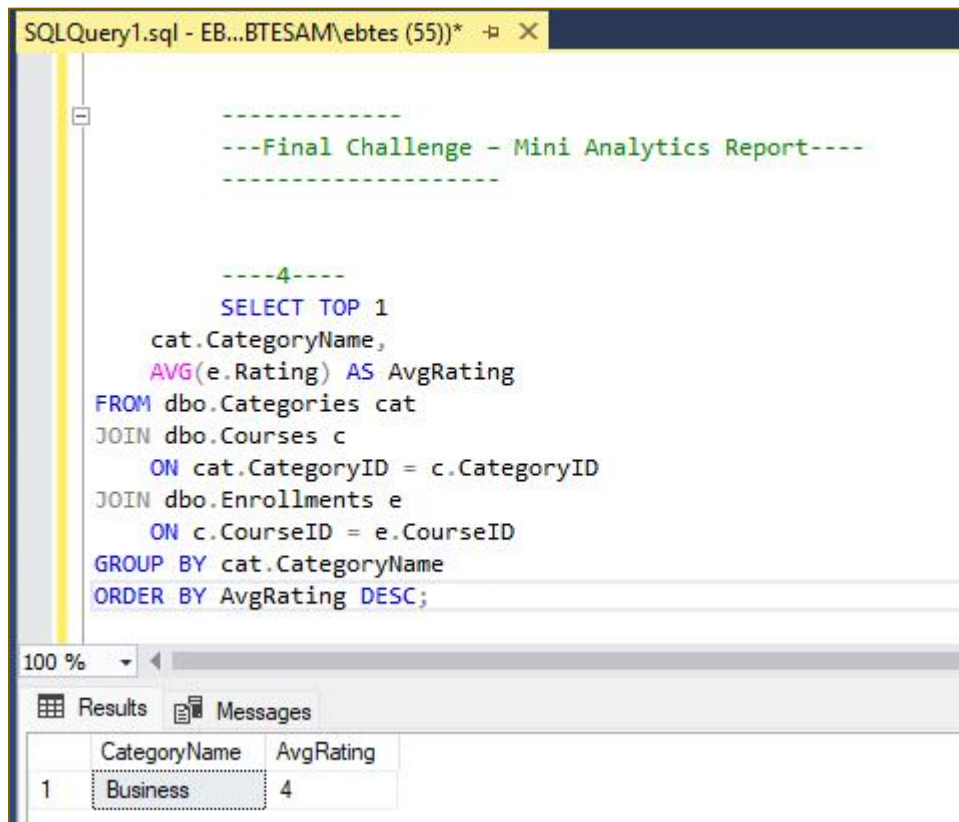
The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is a T-SQL statement that selects the top 1 course by the average completion rate. The query text is as follows:

```
-----  
---Final Challenge - Mini Analytics Report---  
-----  
  
-----3-----  
SELECT TOP 1  
    c.Title AS CourseTitle,  
    AVG(e.CompletionPercent) AS AvgCompletion  
FROM    dbo.Courses c  
JOIN    dbo.Enrollments e  
        ON c.CourseID = e.CourseID  
GROUP BY c.Title  
ORDER BY AvgCompletion ASC;
```

Below the query editor, the "Results" tab is active, displaying a single row of data:

	CourseTitle	AvgCompletion
1	JavaScript Advanced	45

4. Category with highest average rating.

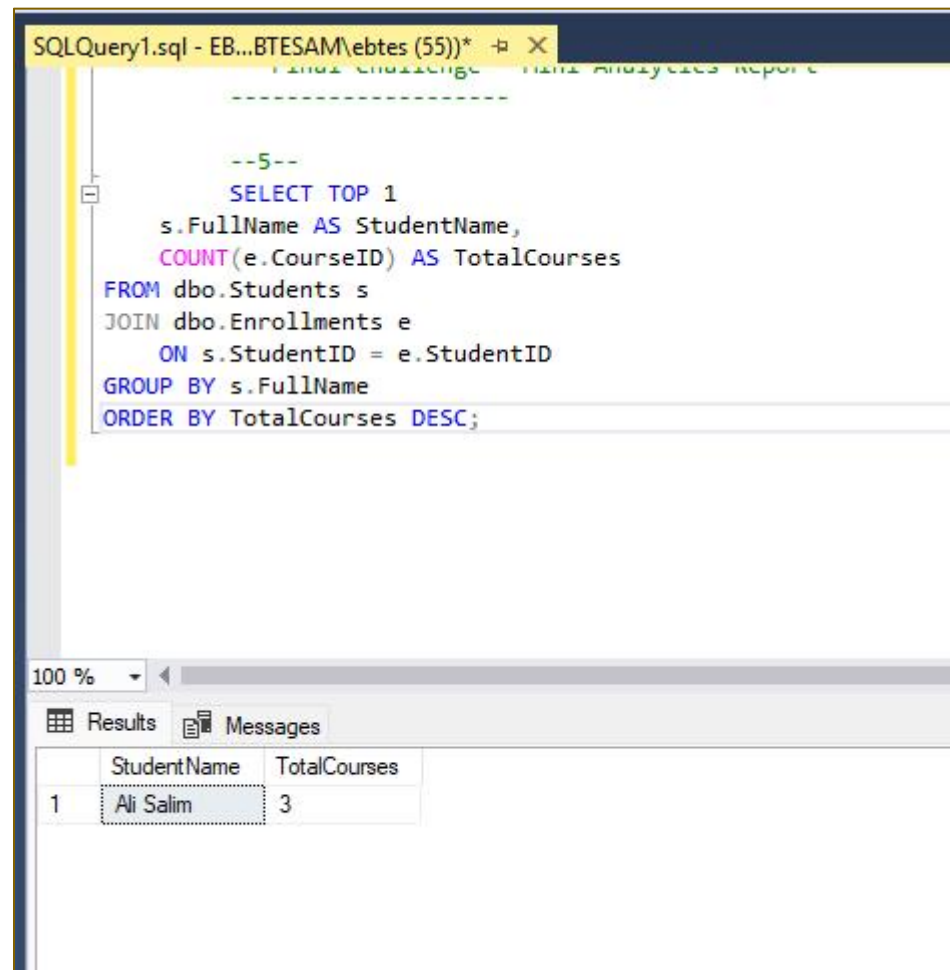


The screenshot shows a SQL query window titled "SQLQuery1.sql - EB...BTESAM\ebtes (55))". The query is a T-SQL statement designed to find the category with the highest average rating. It includes a header comment, a subquery identifier, and a SQL query using JOINs and aggregate functions. Below the query, the "Results" tab is active, displaying a single row of data in a table format.

```
-----  
---Final Challenge - Mini Analytics Report---  
-----  
  
-----4-----  
SELECT TOP 1  
    cat.CategoryName,  
    AVG(e.Rating) AS AvgRating  
FROM    dbo.Categories cat  
JOIN    dbo.Courses c  
    ON  cat.CategoryID = c.CategoryID  
JOIN    dbo.Enrollments e  
    ON  c.CourseID = e.CourseID  
GROUP BY cat.CategoryName  
ORDER BY AvgRating DESC;
```

	CategoryName	AvgRating
1	Business	4

5. Student enrolled in most courses.



The screenshot shows a SQL Server Enterprise Manager window with a query editor and a results pane. The query editor displays a SQL query that selects the top student by the number of courses enrolled in. The results pane shows a single row for 'Ali Salim' with 3 courses.

```
SQLQuery1.sql - EB...BTESAM\ebtes (55))* X
-----
--5--
SELECT TOP 1
  s.FullName AS StudentName,
  COUNT(e.CourseID) AS TotalCourses
FROM dbo.Students s
JOIN dbo.Enrollments e
  ON s.StudentID = e.StudentID
GROUP BY s.FullName
ORDER BY TotalCourses DESC;
```

100 %

Results Messages

	StudentName	TotalCourses
1	Ali Salim	3